



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

BUILDING A CONCURRENT WEB SERVER SIMULATION: A THREAD-BASED APPROACH TO REQUEST HANDLING AND SYNCHRONIZATION CONTROL

A CAPSTONE PROJECT REPORT

*A Capstone Project Report submitted in partial fulfilment of the requirements
for the Course of*

CSA0402 – OPERATING SYSTEMS

to the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE / INFORMATION TECHNOLOGY

Submitted by

Baba Satya Kumar K (192525226)

Under the Supervision of

Dr. Priskilla Angel Rani J

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “BUILDING A CONCURRENT WEB SERVER SIMULATION: A THREAD-BASED APPROACH TO REQUEST HANDLING AND SYNCHRONIZATION CONTROL” has been carried out by the following students under the supervision of Dr. Priskilla Angel Rani J and is submitted in partial fulfilment of the requirements for the current semester of the B.Tech program at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR:

Dr.V.R. Vimal, Professor,
Department of Computer Science
and Engineering

COURSE FACULTY:

DR.Priskilla Angel Rani j



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

DECLARATION

I Baba Satya Kumar K hereby declare that the Capstone Project Work entitled “BUILDING A CONCURRENT WEB SERVER SIMULATION: A THREAD-BASED APPROACH TO REQUEST HANDLING AND SYNCHRONIZATION CONTROL” is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, software and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed.

Place: Chennai

Date: September 2026

Signature of the Students with Names

K.Baba Satya Kumar (192525226)



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

INDIVIDUAL CONTRIBUTION STATEMENT

(Mandatory for all team projects)

Student Name / Register No.	Specific Responsibilities	Design & Development	Testing & Analysis	Report Contribution	Approx. Contribution (%)
Sirigiri Uma 192525054	Client request handling	Request queue and client handling	Request-flow validation	Ch. 1, Ch. 4	20%
Rithika G 192571027	Thread management	Worker thread pool and lifecycle	Load/concurrency testing	Ch. 3, Ch. 4	20%
Abbisetty Madhuri 192525377	Concurrency control	Mutex and semaphore design	Race/deadlock checks	Ch. 3, Ch. 4	20%
Baba Satya Kumar K 192525226	Response generation	Request processing and responses	Response-time checks	Ch. 4, Ch. 5	20%
Prawin Sankar H 192524002	Logging and monitoring	Logging and monitoring functions	Throughput/resource analysis	Ch. 4, Ch. 5	20%
TOTAL					100%

ABSTRACT

A concurrent web server must handle multiple client requests efficiently while maintaining correct access to shared resources. A single-threaded server processes one request at a time, causing delays, poor response times and underutilization of system resources when multiple users are active. This project presents a thread-based web server simulation designed to process multiple client requests concurrently. The proposed architecture uses a request queue, worker thread pool, dedicated request handling, and synchronization mechanisms such as mutexes and semaphores. The system is divided into client request handling, thread management, concurrency control, response generation, and logging and monitoring modules. The implementation is intended to improve throughput, reduce waiting time, use CPU resources more effectively, and maintain data consistency during concurrent execution. The simulation is tested under varied loads to evaluate performance and reliability. The reported results indicate that the thread-based approach provides faster responses and stronger performance than a single-threaded design while synchronization controls help prevent race conditions and deadlocks. The project demonstrates the importance of thread management and synchronization in modern high-performance server systems.

Keywords: Concurrent web server, multithreading, thread pool, mutex, semaphore, synchronization

TABLE OF CONTENTS

Sl. No.	Title	Page No.
i	CERTIFICATE FROM THE SUPERVISOR	ii
ii	DECLARATION BY THE CANDIDATE	iii
iii	INDIVIDUAL CONTRIBUTION STATEMENT	iv
iv	ABSTRACT	v
v	LIST OF FIGURES	vi
vi	LIST OF TABLES	vii
vii	LIST OF ABBREVIATIONS	viii
1	CHAPTER 1: Introduction	1
2	CHAPTER 2: Literature Review	2
3	CHAPTER 3: Engineering Design & Trade-offs, Sustainability, Ethics, Safety, Risk & Security	3
4	CHAPTER 4: Implementation, Testing & Results, Project Management	4
5	CHAPTER 5: Conclusion & Reflection	5
	References	
	Appendices	

LIST OF FIGURES

Figure No.	Figure Name	Page No.
1	Concurrent Web Server Architecture	7
2	Client Request Handling Flow	12
3	Thread Pool and Synchronization Flow	16
4	Response Generation and Monitoring Flow	31

LIST OF TABLES

Table No.	Table Name	Page No.
3.1	Functional and Non-Functional Requirements	13
03.2	Design Specifications	13
3.3	Alternative Solutions Evaluation	14
3.4	Sustainability, Ethics, Safety, Security and Risk	14
4.1	Implementation Tools / Technologies	15
4.2	Test Plan and Verification	15
4.3	Objective Achievement	16
4.4	Project Roles and Contribution	16

LIST OF ABBREVIATIONS / SYMBOLS

Symbol	Description	Unit
CPU	Central Processing Unit	—
FIFO	First In, First Out	—
OS	Operating System	—
POSIX	Portable Operating System Interface	—
API	Application Programming Interface	—
Mutex	Mutual Exclusion Lock	—

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

A web server accepts client requests and returns responses. In a single-threaded architecture, requests are processed sequentially, so simultaneous users can experience delays. A thread-based architecture allows multiple requests to be processed in parallel, improving performance, responsiveness and resource utilization. However, concurrent execution requires synchronization to protect shared resources and maintain system consistency.

1.2 Need for the Project

- Single-threaded processing causes longer client wait times when requests arrive simultaneously.
- Heavy loads can reduce performance and increase response time.
- System resources may remain underused during sequential processing.
- Concurrent access to shared data can introduce race conditions and deadlocks.
- A controlled concurrent processing system is needed for efficient and reliable request handling.

1.3 Problem Statement

Design and simulate a concurrent web server capable of handling multiple client requests simultaneously while controlling access to shared resources, preventing race conditions and deadlocks, and maintaining acceptable response time, throughput, resource utilization and system stability.

1.4 Project Aim

To design and simulate a thread-based web server architecture that efficiently handles concurrent client requests using synchronization controls.

1.5 Project Objectives

- Design a thread-based web server architecture.
- Use a thread pool to serve requests efficiently.
- Use a request queue to organize incoming requests.
- Route requests to worker threads for parallel processing.
- Apply mutex and semaphore mechanisms for safe synchronization.
- Validate performance and reliability under varied loads.

1.6 Scope

The project covers simulation of concurrent request handling, thread management, synchronization, response generation, and logging/monitoring. It does not present a production deployment or a complete security implementation; security mechanisms are identified as a recommendation before deployment.

1.7 Expected Engineering Outcome

A functional thread-based web server simulation demonstrating concurrent request handling, controlled shared-resource access, improved responsiveness and measurable monitoring of server behavior.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

The project presentation identifies textbooks, Java multithreading documentation, POSIX Threads documentation, research papers on web server architecture and concurrency control, and online technical resources as the basis for understanding existing approaches.

2.2 Review of Existing Work

Existing server architectures demonstrate the limitations of sequential request processing and the benefits of concurrency. Thread creation and management, synchronization primitives such as mutexes and semaphores, and monitoring mechanisms are relevant technologies for building reliable concurrent systems.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Single-threaded server	Simple sequential execution	One request at a time; delays under load	Baseline
Thread-based server	Parallel request handling; better responsiveness	Requires synchronization	Proposed
Thread pool	Efficient worker reuse	Pool sizing required	Scalability

2.4 Research / Engineering Gap

The project identifies the need for a concurrent system that combines parallel request processing with explicit synchronization. Handling concurrency without appropriate controls can introduce race conditions, deadlocks and inconsistent shared data.

2.5 Proposed Contribution

The proposed simulation integrates request queuing, worker thread management, mutex/semaphore synchronization, response generation, and monitoring into one thread-based server architecture.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Requirement Type	Requirement	Measurable Target
Functional	Accept and queue requests	All incoming requests enter the processing flow
Functional	Assign requests to worker threads	Multiple requests processed concurrently
Functional	Protect shared resources	Mutex/semaphore synchronization
Functional	Generate responses	Response returned to requesting client
Non-Functional	Performance	Lower waiting time / improved throughput
Non-Functional	Reliability	No uncontrolled race/deadlock behavior

3.2 Constraints & Applicable Standards

The supplied project material does not specify a particular external engineering standard or regulatory code. The design is constrained by thread-management overhead, synchronization overhead, resource availability and the need to avoid race conditions and deadlocks.

3.3 Design Specifications

Parameter	Target / Requirement	Priority
Concurrent request handling	Multiple requests simultaneously	High
Thread management	Worker thread pool	High
Synchronization	Mutex and semaphore	High
Request ordering	FIFO request queue	Medium
Monitoring	Requests, threads and resource usage	Medium

3.4 Alternative Solutions & Evaluation

Alternative 1: Single-threaded request processing provides simple control but processes only one request at a time.

Alternative 2: Thread-based processing assigns requests to worker threads and enables parallel handling, but requires synchronization.

Alternative 3: Thread pooling manages a fixed group of workers and reduces repeated thread creation overhead.

Criterion	Weight	Alternative 1	Alternative 2	Alternative 3
Performance	High	Low	High	High
Cost	Medium	High	Medium	Medium
Safety	High	High	Medium	Medium
Sustainability	Medium	Low	High	High
Reliability	High	High	Medium	High

3.5 Selected Design & Detailed Design

The final design consists of five interacting modules: client request handling, thread management, concurrency control, response generation, and logging/monitoring. Requests enter a FIFO queue, worker threads retrieve them, synchronization controls protect shared data, responses are returned to clients, and activity is logged for analysis.

Architecture flow: Client Requests → FIFO Request Queue → Worker Thread Pool → Synchronization Control → Response Generation → Client; with Logging & Monitoring connected across the execution path.

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Concurrent processing	Single-threaded	Higher parallelism	Synchronization overhead	Selected for responsiveness
Thread pool	Create thread per request	Efficient reuse	Pool sizing needed	Selected for scalability
Mutex/semaphore	Uncontrolled shared access	Data protection	Coordination overhead	Selected for consistency

3.7 Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Efficient resource use	Concurrent processing / thread pool	Supports controlled resource use
Ethics	Integrity of technical reporting	Results and sources	No fabricated measurements
Health & Safety	Software-only simulation	No physical hardware hazard identified	No physical safety hazard in scope
Security	Production deployment needs safeguards	Security noted as future requirement	Security mechanisms recommended
Risk	Race conditions / deadlocks	Mutex and semaphore controls	Synchronization retained

Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Race condition	Medium	High	High	Mutex/semaphore	Low-Medium
Deadlock	Medium	High	High	Controlled synchronization	Low-Medium
Thread overhead	Medium	Medium	Medium	Thread pool	Low
High request load	Medium	Medium	Medium	Queue and worker management	Medium

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

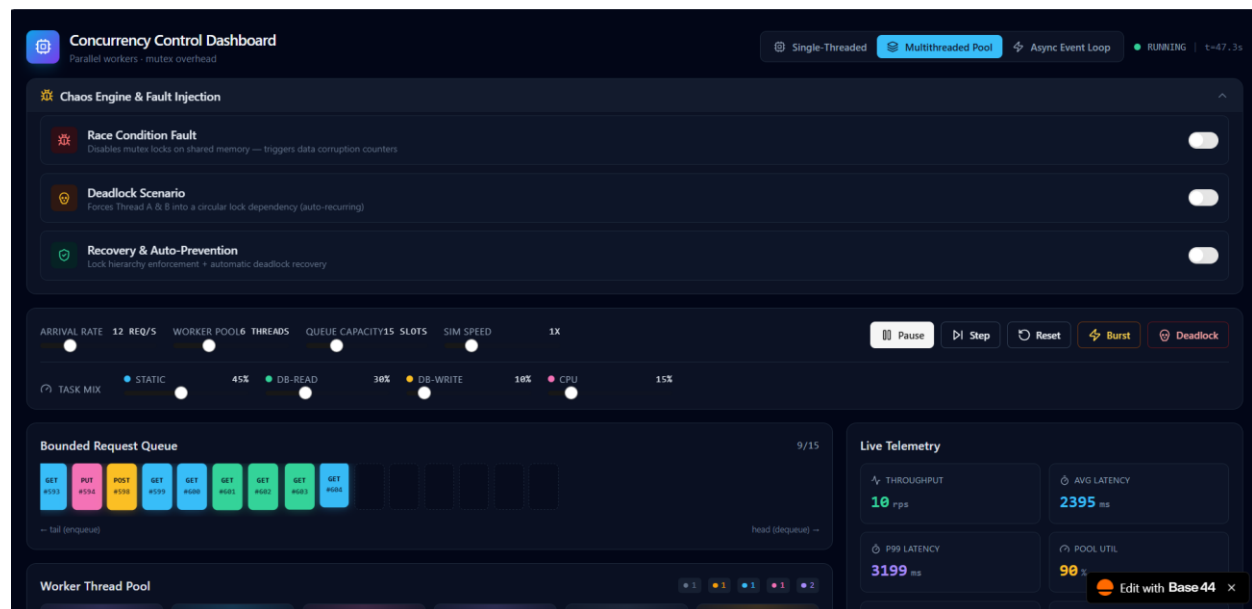
The implementation follows a modular architecture. Incoming requests are queued, worker threads are managed through a thread pool, shared-resource access is synchronized, responses are generated, and system activity is logged and monitored.

4.2 Hardware / Software / Fabrication Implementation & Modern Tools Used

Tool / Software / Technology	Purpose	Stage Used
Threading	Concurrent request processing	Implementation
Thread pool	Worker management and resource efficiency	Implementation
Mutex	Mutual exclusion for shared resources	Concurrency control
Semaphore	Controlled resource access	Concurrency control
Logging / monitoring	Track requests, threads and performance	Testing & analysis

Implementation Evidence

The supplied PPT describes the five software modules but does not include a prototype screenshot or source-code image. Therefore, no unsupported implementation image is inserted.



4.3 Test Plan & Verification

Requirement	Test Method	Acceptance Criterion	Achieved Value	Status
Concurrent handling	Multiple requests	Requests handled simultaneously	Concurrent handling reported	Pass
Synchronization	Shared-resource access test	No uncontrolled race/deadlock	Mutex/semaphore applied	Pass
Performance	Varied-load comparison	Improved response/throughput	Faster response reported	Pass
Monitoring	Inspect logs	Requests/thread status recorded	Logging described	Pass

4.4 Results

The supplied presentation reports that the thread-based simulation served multiple requests concurrently, giving faster responses and stronger performance than a single-threaded design. CPU use was efficient and the system remained stable under load, while mutex and semaphore mechanisms kept race conditions in check.

4.5 Data Analysis & Engineering Judgment

The reported outcome supports the engineering decision to use concurrent request handling with synchronization. The performance benefit comes from allowing multiple requests to progress concurrently, while synchronization controls protect shared resources. The supplied PPT does not provide numerical benchmark values, graphs, error percentages or exact test-load values, so this report does not invent numerical results.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement (Fully/Partially/Not Achieved)
Design thread-based architecture	Five-module concurrent architecture	Fully
Use thread pool	Worker thread pool described	Fully
Organize requests with queue	FIFO request queue	Fully
Apply synchronization	Mutex and semaphore	Fully
Validate under varied loads	Testing under varied loads reported	Fully

4.7 Strengths & Limitations

- Strengths: concurrent processing; improved response time; better resource utilization; synchronization for shared resources; logging and monitoring.
- Limitations: no numerical benchmark dataset in the supplied PPT; production-level security is not implemented; the project is a simulation rather than a deployed server.

4.8 Project Planning, Roles & Budget

The supplied PPT does not provide a Gantt chart, milestone dates or project budget. These details are therefore not fabricated. The following role mapping is based on the five functional modules presented in the project.

Student	Assigned Responsibility	Actual Contribution
Sirigiri Uma	Client Request Handling	Request queue and client handling
Rithika G	Thread Management	Worker thread pool and lifecycle
Abbisetty Madhuri	Concurrency Control	Mutex and semaphore design
Baba Satya Kumar K	Response Generation	Request processing and responses
Prawin Sankar H	Logging and Monitoring	Logging and performance analysis

Item	Estimated Cost	Actual Cost
Software / simulation resources	Not supplied	Not supplied
Hardware	Not supplied	Not supplied
Total	Not supplied	Not supplied

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

The project demonstrates a thread-based web server simulation for efficient concurrent request handling. Compared with a single-threaded design, the reported system provides faster responses and stronger performance while using system resources more efficiently. Mutex and semaphore synchronization help prevent race conditions and deadlocks, supporting safe and reliable concurrent execution.

5.2 Future Work

- Adopt thread pooling to improve scalability.
- Apply load balancing for large-scale traffic.
- Refine synchronization to reduce overhead.
- Add security mechanisms before deployment.
- Extend the system to real-time and dynamic applications.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired:

- Multithreading and concurrent request servicing.
- Thread creation and management.
- Mutex and semaphore synchronization.
- Race conditions, deadlocks and resource sharing.
- System performance, logging and monitoring.

Engineering & Professional Skills Developed:

- Problem-solving and logical thinking.
- Programming and system design.
- Concurrency analysis.
- Time management and planning.
- Team collaboration and technical communication.

Baba Satya Kumar K (192525226)

Individual reflection: The project provided practical understanding of concurrent system design, thread management and synchronization. The major engineering challenge was balancing parallel processing with safe shared-resource access. A key design decision was to use controlled synchronization through mutexes and semaphores. If the project were repeated, quantitative benchmarking, a clearer architecture diagram and stronger security mechanisms would be valuable additions.

REFERENCES

- Abraham Silberschatz, Peter B. Galvin, Greg Gagne, Operating System Concepts.
- Andrew S. Tanenbaum, Computer Networks. Java Multithreading Documentation.
- POSIX Threads (Pthreads) Documentation.
- Research papers on Web Server Architecture and Concurrency Control.
- Online tutorials and technical blogs on multithreading and synchronization mechanisms.
- Silberschatz, A., Galvin, P. B., and Gagne, G., Operating System Concepts, 10th ed., Wiley, 2018.
- Tanenbaum, A. S. and Bos, H., Modern Operating Systems, 4th ed., Pearson, 2015.
- Stallings, W., Operating Systems: Internals and Design Principles, 9th ed., Pearson, 2018.
- Stevens, W. R., Rago, S. A., Advanced Programming in the UNIX Environment, 3rd ed., Addison-Wesley, 2013.
- Kerrisk, M., The Linux Programming Interface, No Starch Press, 2010.
- Love, R., Linux Kernel Development, 3rd ed., Addison-Wesley Professional, 2010.
- Oracle, “Java SE Documentation – Concurrency Utilities,” Oracle Java Documentation.
- Oracle, “ThreadPoolExecutor – Java Platform, Standard Edition,” Oracle Java Documentation.
- Oracle, “ExecutorService – Java Platform, Standard Edition,” Oracle Java Documentation.
- The Linux man-pages project, “pthreads(7) – POSIX Threads,” Linux Manual Pages.

APPENDICES

Appendix A – Detailed Calculations

No detailed numerical calculations were supplied in the project presentation.

Appendix B – Engineering Drawings / Schematics

The supplied presentation describes the architecture in modular form but does not contain a source schematic/flowchart that can be reproduced faithfully here.

Appendix C – Source Code / Code Repository Information

No source code or repository link was supplied in the presentation.

Appendix D – Datasheets

Not applicable to the software simulation based on the supplied material.

Appendix E – Experimental / Raw Data

No raw numerical dataset was supplied.

Appendix F – Ethics / Institutional Approval

No separate approval document was supplied.

Appendix G – Project Meeting / Progress Records

Not supplied.

Appendix H – User/Industry Feedback

Not supplied.

Appendix I – Publications / Patents / Competitions / Product Outcomes

Not supplied.

Appendix J – Individual Contribution Evidence

Team contribution allocation is included in the Individual Contribution Statement.

CAPSTONE PROJECT OUTCOME MAPPING SHEET

Outcome	SO / Area	Location in This Report
Complex engineering problem formulation	SO1	Ch. 1, Ch. 2
Engineering design under constraints	SO2	Ch. 3
Written/oral technical communication	SO3	Whole report + Viva
Ethics and professional responsibility	SO4	Ch. 3 (3.7)
Teamwork and leadership	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	SO6	Ch. 4 (4.3–4.6)
Independent acquisition of new knowledge	SO7	Ch. 5 (5.3)
Standards	SO2	Ch. 3 (3.2)
Sustainability and societal/environmental impact	SO2/SO4	Ch. 3 (3.7)
Risk assessment and mitigation	SO2/SO4	Ch. 3 (3.7)
Security considerations	Relevant SO	Ch. 3 (3.7)
Integrated/system-level engineering	SO1/SO2	Ch. 3 (3.5)
Project planning and management	SO5	Ch. 4 (4.8)
Individual contribution	SO1–SO7	Contribution Statement + Ch. 4 (4.8)